SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)

Activity Tool: Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS Total Marks: 50

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	**5**
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	**5**
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	**5**
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	**5**
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	**5**

6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	**5**
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	**5**

Code of Conduct:

I A Mohammad Naheed -192525336 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A Mohammad Naheed

Q1. Analyze the case: A food-delivery application stores Delivery(OrderNo, UserID, UserName, ItemID, ItemName, Quantity, UnitPrice). Identify FDs and normalize to 3NF.

Functional Dependencies
• UserID → UserName
• ItemID → ItemName, UnitPrice
• OrderNo → UserID, ItemID, Quantity

Analysis
UserName depends on UserID, while UserID is determined by OrderNo. Similarly, ItemName and UnitPrice depend on ItemID rather than directly on the order. These transitive dependencies violate 3NF.

3NF Decomposition
USER(UserID, UserName)
ITEM(ItemID, ItemName, UnitPrice)
DELIVERY(OrderNo, UserID, ItemID, Quantity)

The decomposition removes transitive dependencies and ensures that every non-key attribute depends directly on the key of its relation.

Executed Output (MySQL)

OrderNo	UserID	UserName	ItemID	ItemName	Quantity	UnitPrice
O01	U01	Rahul	I01	Burger	2	150
O02	U01	Rahul	I02	Pizza	1	300
O03	U02	Priya	I01	Burger	3	150

Fig. Q1 — 3NF decomposition executed in MySQL.

Q2. A clinic maintains MedicalRecord(RecordID, PatientName, DoctorID, DoctorName, Specialty, RoomID, RoomName). Identify anomalies and normalize to BCNF.

Functional Dependencies
• RecordID → PatientName, DoctorID, RoomID
• DoctorID → DoctorName, Specialty
• RoomID → RoomName

Anomalies
• Insertion: a doctor cannot be stored until a patient record is created.
• Deletion: removing the last record for a doctor may remove the doctor's specialty

information.
• Update: changing a doctor's specialty requires multiple rows to be updated.

BCNF Decomposition
DOCTOR(DoctorID, DoctorName, Specialty)
ROOM(RoomID, RoomName)
PATIENT_RECORD(RecordID, PatientName, DoctorID, RoomID)

Each determinant becomes a key in its corresponding relation, satisfying BCNF and reducing insertion, deletion, and update anomalies.

Executed Output (MySQL)

```
mysql> SELECT * FROM MEDICAL_RECORD_UNNORM;
+----------+-------------+----------+------------+------------+--------+--------------+
| RecordID | PatientName | DoctorID | DoctorName | Specialty  | RoomID | RoomName     |
+----------+-------------+----------+------------+------------+--------+--------------+
| R01      | Arun        | D01      | Dr. Kumar  | Cardiology | RM01   | General Ward |
| R02      | Meena       | D02      | Dr. Priya  | Neurology  | RM02   | Special Ward |
| R03      | Karthik     | D01      | Dr. Kumar  | Cardiology | RM01   | General Ward |
+----------+-------------+----------+------------+------------+--------+--------------+
3 rows in set (0.00 sec)
```

Fig. Q2 — BCNF decomposition executed in MySQL.

Q3. Case study: A college stores Registration(StudentNo, StudentName, SubjectCode, SubjectName, Lecturer, Grade). Analyze FDs and apply 2NF decomposition.

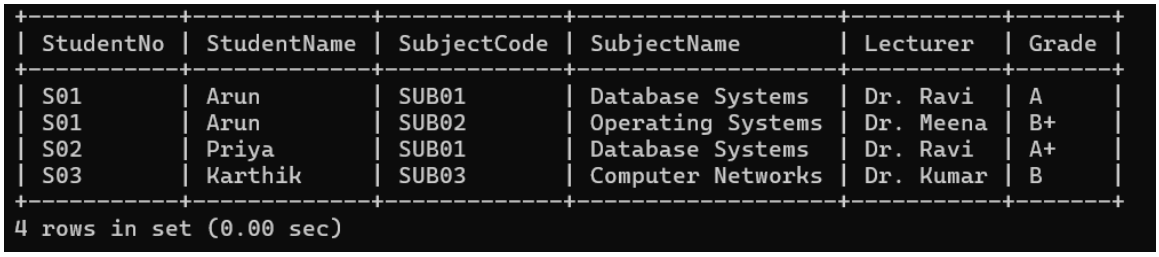
Functional Dependencies
• StudentNo → StudentName
• SubjectCode → SubjectName, Lecturer
• StudentNo, SubjectCode → Grade

Candidate Key
The candidate key is {StudentNo, SubjectCode}. StudentName depends only on StudentNo and SubjectName/Lecturer depend only on SubjectCode. These are partial dependencies and therefore violate 2NF.

2NF Decomposition
STUDENT(StudentNo, StudentName)
SUBJECT(SubjectCode, SubjectName, Lecturer)
REGISTRATION(StudentNo, SubjectCode, Grade)

Grade remains with the complete composite key, while the partial dependencies are removed.

Executed Output (MySQL)



StudentNo	StudentName	SubjectCode	SubjectName	Lecturer	Grade
S01	Arun	SUB01	Database Systems	Dr. Ravi	A
S01	Arun	SUB02	Operating Systems	Dr. Meena	B+
S02	Priya	SUB01	Database Systems	Dr. Ravi	A+
S03	Karthik	SUB03	Computer Networks	Dr. Kumar	B

4 rows in set (0.00 sec)

Fig. Q3 — 2NF decomposition executed in MySQL

Q4. Given the railway reservation case: Reservation(TrainNo, PassengerNo, PassengerName, CoachSeat, StartStation, EndStation, TravelDate). Normalize up to BCNF.

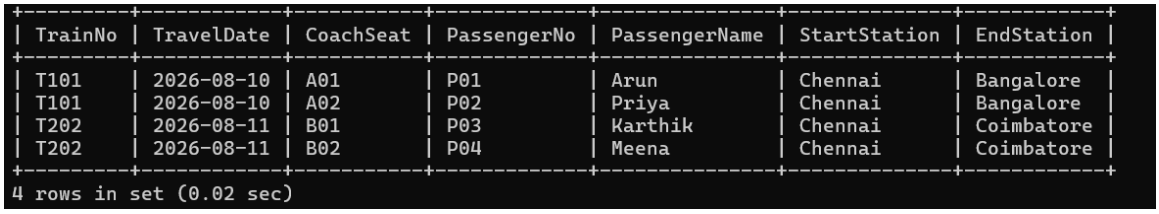
Functional Dependencies
• TrainNo → StartStation, EndStation
• PassengerNo → PassengerName
• TrainNo, TravelDate, CoachSeat → PassengerNo

Candidate Key
{TrainNo, TravelDate, CoachSeat} is the candidate key because a particular seat on a train on a given date is assigned to one passenger.

BCNF Decomposition
TRAIN(TrainNo, StartStation, EndStation)
PASSENGER(PassengerNo, PassengerName)
RESERVATION(TrainNo, TravelDate, CoachSeat, PassengerNo)

The determinants in the decomposed relations are keys, so the resulting design satisfies BCNF.

Executed Output (MySQL)



TrainNo	TravelDate	CoachSeat	PassengerNo	PassengerName	StartStation	EndStation
T101	2026-08-10	A01	P01	Arun	Chennai	Bangalore
T101	2026-08-10	A02	P02	Priya	Chennai	Bangalore
T202	2026-08-11	B01	P03	Karthik	Chennai	Coimbatore
T202	2026-08-11	B02	P04	Meena	Chennai	Coimbatore

4 rows in set (0.02 sec)

Fig. Q4 — BCNF decomposition executed in MySQL

Q5. Analyze an organizational payroll schema for a large company. Identify functional and transitive dependencies and normalize to 3NF.

Schema
PAYROLL(StaffID, StaffName, DivisionID, DivisionName, Office, BasicPay)

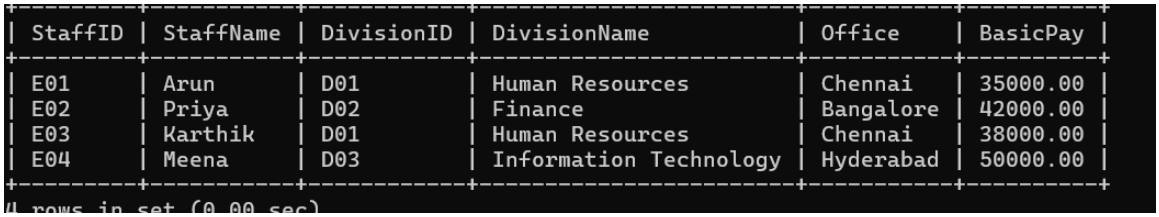
Functional Dependencies
• StaffID → StaffName, DivisionID, BasicPay
• DivisionID → DivisionName, Office

Transitive Dependency
StaffID → DivisionID and DivisionID → DivisionName, Office together imply StaffID → DivisionName, Office transitively. Hence the original relation violates 3NF.

3NF Decomposition
DIVISION(DivisionID, DivisionName, Office)
STAFF(StaffID, StaffName, DivisionID, BasicPay)

Division details now depend directly on DivisionID, eliminating the transitive dependency.

Executed Output (MySQL)



StaffID	StaffName	DivisionID	DivisionName	Office	BasicPay
E01	Arun	D01	Human Resources	Chennai	35000.00
E02	Priya	D02	Finance	Bangalore	42000.00
E03	Karthik	D01	Human Resources	Chennai	38000.00
E04	Meena	D03	Information Technology	Hyderabad	50000.00

4 rows in set (0.00 sec)

Fig. Q5 — 3NF decomposition executed in MySQL.

Q6. Case: DigitalLibrary(MemberNo, MemberName, PublicationID, Title, Writer, Topic, IssueDate). Identify MVDs and decompose to 4NF.

Why MVDs arise
A publication may have several writers and several topics independently. Keeping both independent lists in one relation creates unnecessary combinations of writers and topics.

Multi-valued Dependencies
• PublicationID →→ Writer
• PublicationID →→ Topic

4NF Decomposition
PUBLICATION(PublicationID, Title)
PUBLICATION_WRITER(PublicationID, Writer)
PUBLICATION_TOPIC(PublicationID, Topic)
ISSUE(MemberNo, PublicationID, IssueDate)

The independent multi-valued facts are separated, removing the cross-product redundancy and satisfying 4NF.

Executed Output (MySQL)

```
+----------+---------------+--------------------+--------+------------+------------+
| MemberNo | PublicationID | Title              | Writer | Topic      | IssueDate  |
+----------+---------------+--------------------+--------+------------+------------+
| M01      | P01           | Database Basics    | Meena  | Education  | 2026-08-01 |
| M01      | P01           | Database Basics    | Meena  | Technology | 2026-08-01 |
| M01      | P01           | Database Basics    | Ravi   | Education  | 2026-08-01 |
| M01      | P01           | Database Basics    | Ravi   | Technology | 2026-08-01 |
| M02      | P02           | Computer Networks  | Kumar  | Networking | 2026-08-05 |
+----------+---------------+--------------------+--------+------------+------------+
5 rows in set (0.00 sec)
```

Fig. Q6 — 4NF decomposition executed in MySQL.

Q7. Analyze a mobile-network billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Schema
MOBILE_BILL(CustomerNo, CustomerName, MobileNo, PackageID, PackageName, UsageID, UsageDate, Minutes, Charge)

Functional Dependencies
• UsageID → CustomerNo, UsageDate, Minutes, Charge
• CustomerNo → CustomerName, MobileNo, PackageID
• MobileNo → CustomerNo
• PackageID → PackageName

Candidate Keys
• UsageID is a candidate key and can serve as the primary key of the billing relation.
• CustomerNo and MobileNo are candidate keys for the customer entity after decomposition.
• PackageID is the candidate key of the package entity.

This key analysis identifies the determinants that must be considered before normalization.

Executed Output (MySQL)

```
+---------+------------+--------------+------------+-----------+-------------+------------+---------+--------+
| UsageID | CustomerNo | CustomerName | MobileNo   | PackageID | PackageName | UsageDate  | Minutes | Charge |
+---------+------------+--------------+------------+-----------+-------------+------------+---------+--------+
| U01     | C01        | Arun         | 9876500001 | PK01      | Basic       | 2026-08-01 |     120 | 250.00 |
| U02     | C02        | Priya        | 9876500002 | PK02      | Premium     | 2026-08-02 |     250 | 500.00 |
| U03     | C03        | Karthik      | 9876500003 | PK01      | Basic       | 2026-08-03 |     180 | 300.00 |
| U04     | C04        | Meena        | 9876500004 | PK03      | Student     | 2026-08-04 |     100 | 150.00 |
+---------+------------+--------------+------------+-----------+-------------+------------+---------+--------+
4 rows in set (0.00 sec)
```

Fig. Q7 — Candidate key and FD verification executed in MySQL.

Q8. A warehouse management system has Item(ItemCode, ItemName, VendorID, VendorName, Section, Cost, StorageLocation). Apply complete normalization.

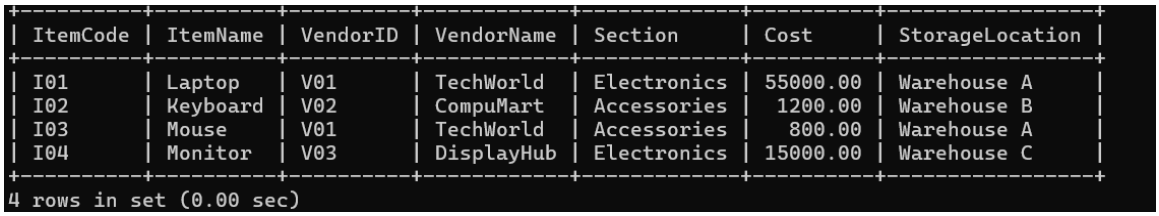
Functional Dependencies
• ItemCode → ItemName, VendorID, Section, Cost, StorageLocation
• VendorID → VendorName

Normal-form walkthrough
• 1NF: all attributes contain atomic values — satisfied.
• 2NF: ItemCode is a single-attribute key, so partial dependency cannot occur — satisfied.
• 3NF: VendorName depends transitively on ItemCode through VendorID — violation.

Complete Decomposition
VENDOR(VendorID, VendorName)
ITEM(ItemCode, ItemName, VendorID, Section, Cost, StorageLocation)

Both relations satisfy BCNF because VendorID and ItemCode are keys of their respective relations.

Executed Output (MySQL)



ItemCode	ItemName	VendorID	VendorName	Section	Cost	StorageLocation
I01	Laptop	V01	TechWorld	Electronics	55000.00	Warehouse A
I02	Keyboard	V02	CompuMart	Accessories	1200.00	Warehouse B
I03	Mouse	V01	TechWorld	Accessories	800.00	Warehouse A
I04	Monitor	V03	DisplayHub	Electronics	15000.00	Warehouse C

4 rows in set (0.00 sec)

Fig. Q8 — Complete normalization executed in MySQL.

Q9. Study the following poorly normalized College ERP schema and propose a fully normalized design up to BCNF with justification.

Poorly Normalized Schema
COLLEGE(StudentNo, StudentName, ProgramID, ProgramName, MentorID, MentorName, Subject, Score)

Functional Dependencies
• StudentNo → StudentName, ProgramID
• ProgramID → ProgramName
• MentorID → MentorName
• StudentNo, Subject → Score, MentorID

Justification
ProgramName depends transitively on StudentNo through ProgramID. MentorName depends on MentorID rather than directly on the complete key. These dependencies cause normalization violations.

BCNF Design
STUDENT(StudentNo, StudentName, ProgramID)
PROGRAM(ProgramID, ProgramName)
MENTOR(MentorID, MentorName)
RESULT(StudentNo, Subject, MentorID, Score)

Each determinant is a key in its own relation, eliminating the identified BCNF violations.

Executed Output (MySQL)

StudentNo	StudentName	ProgramID	ProgramName	Subject	MentorID	MentorName	Score
S01	Arun	P01	Computer Science	DBMS	M01	Dr. Ravi	85
S01	Arun	P01	Computer Science	OS	M02	Dr. Meena	78
S02	Priya	P02	Artificial Intelligence	DBMS	M01	Dr. Ravi	92
S03	Karthik	P01	Computer Science	Networks	M03	Dr. Kumar	80

4 rows in set (0.00 sec)

Fig. Q9 — BCNF redesign executed in MySQL.

Q10. Given a video-on-demand platform schema, identify the join dependency and decompose to 5NF with lossless-join verification.

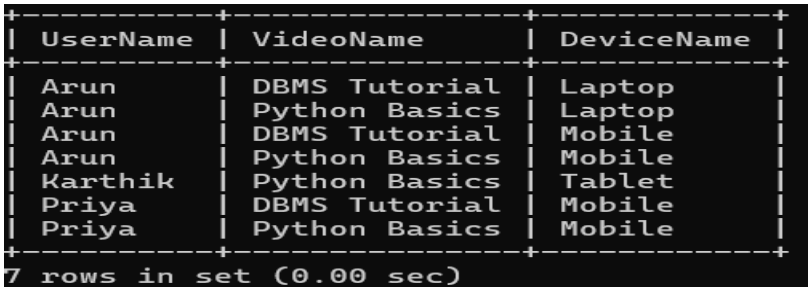
Schema
VIEWING(User, Video, Device)

Business Rule
If a user watches a video, the user uses a device, and that video is supported on the device, the corresponding three-way viewing combination is valid. This represents a three-way join dependency JD(UV, VD, UD).

5NF Decomposition
UV(User, Video)
VD(Video, Device)
UD(User, Device)

Lossless-Join Verification
Joining UV, VD, and UD using their common attributes reconstructs the valid viewing combinations without introducing spurious tuples. The three projections are therefore required to represent the join dependency in 5NF.

Executed Output (MySQL)



UserName	VideoName	DeviceName
Arun	DBMS Tutorial	Laptop
Arun	Python Basics	Laptop
Arun	DBMS Tutorial	Mobile
Arun	Python Basics	Mobile
Karthik	Python Basics	Tablet
Priya	DBMS Tutorial	Mobile
Priya	Python Basics	Mobile

7 rows in set (0.00 sec)

Fig. Q10 — 5NF decomposition and lossless-join verification executed in MySQL.